

第三章 Lesson5

Control

笔记整理自 2026 年 9 月 21 日课程

1. 复习：寻址与 leaq

一般内存操作数写作 $D(R_b, R_i, S)$ ，表示 $\text{Mem}[\text{Reg}[R_b] + S \times \text{Reg}[R_i] + D]$ 。其中 D 是位移量， R_b 是基址寄存器， R_i 是变址寄存器，比例因子 S 只能是 1, 2, 4, 8。例如 `6(%rbx,%rdi,8)` 对应地址 $\%rbx + 8 * \%rdi + 6$ 。

大多数指令把这种写法当作内存地址来访问；`leaq` 是例外：它只计算有效地址，不读内存。

```
leaq 6(%rbx,%rdi,8), %rax    # rax = rbx + rdi*8 + 6
leaq (%rbx,%rbx,2), %rax     # rax = rbx * 3
```

编译器也常把 `leaq` 用作算术指令：一次完成加法与有限倍数乘法，并且不改动条件码。常见后缀 `b/w/l/q` 分别表示字节、16 位、32 位和 64 位操作；反汇编常省略后缀。

2. 控制流的机器视角

高级语言中的 `if`、循环和 `switch` 最终都由跳转实现。处理器需要保存：

- `%rip`：下一条将要执行的指令地址；
- `%rsp`：运行时栈顶地址；
- 通用寄存器：暂存数据与函数参数；
- 条件码：最近一次算术或逻辑测试的状态。

例如 C 代码

```
if (x) op1();
else  op2();
```

可对应为

```
testl %edi, %edi    # 检查 x 是否为 0
je     .L2          # x == 0 时跳到 else
call  op1
jmp    .L1          # 跳过 else
.L2:
call  op2
.L1:
```

所以汇编层面本质上是标签与 `goto`；结构化控制流是编译器生成这种跳转序列的规则。

3. 条件码 (condition codes)

条件码是单比特寄存器，常用的四个为：

名称	含义	主要用途
CF	Carry Flag, 进位标志	无符号运算的溢出或借位
ZF	Zero Flag, 零标志	结果是否为 0
SF	Sign Flag, 符号标志	结果按有符号数解释是否为负
OF	Overflow Flag, 溢出标志	补码有符号运算是否溢出

3.1. 算术指令的隐式设置

以 `addq Src, Dest` 为例，令 $t = a + b$ 。指令写回结果，同时更新条件码：

- $ZF=1$ 当且仅当 $t = 0$;
- $SF=1$ 当且仅当 $t < 0$ (按有符号数解释);
- $CF=1$ 表示最高位产生进位，可理解为无符号加法溢出；减法时它反映借位；
- $OF=1$ 表示有符号补码溢出。加法中，两个同号数相加却得到异号结果时溢出：

$$(a > 0 \wedge b > 0 \wedge t < 0) \vee (a < 0 \wedge b < 0 \wedge t \geq 0)$$

。

`leaq` 不会设置条件码，这使它适合做不应影响后续判断的地址或算术计算。

3.2. `cmp` 与 `test` 的显式设置

```
cmpq Src2, Src1      # 按 Src1 - Src2 计算条件码，不保存结果
testq Src2, Src1     # 按 Src1 & Src2 计算条件码，不保存结果
```

注意 AT&T 语法的操作数顺序：`cmpq b, a` 等价于考察 $a - b$ 。因此在

```
cmpq %rsi, %rdi      # 比较 x:y, 即计算 x - y 的条件码
```

之后，可以依据 `%rdi` 中的 x 与 `%rsi` 中的 y 的关系跳转。`testq %rax,%rax` 很常见：它不改变 `%rax`，但能测试该值是否为零或负数。`test` 也适合与掩码结合，例如测试某一位是否为 1。

3.3. 从条件码得到布尔值：`setX`

`setX` 指令根据条件码把目标寄存器的低 1 字节置为 0 或 1，其余字节不变。常用条件如下：

指令	条件	含义
<code>sete / setne</code>	$ZF / \sim ZF$	相等 / 不相等
<code>sets / setns</code>	$SF / \sim SF$	负 / 非负
<code>setg / setge</code>	$\sim(SF \wedge OF) \wedge \sim ZF / \sim(SF \wedge OF)$	有符号大于 / 大于等于
<code>setl / setle</code>	$SF \wedge OF / (SF \wedge OF) \mid ZF$	有符号小于 / 小于等于
<code>seta / setb</code>	$\sim CF \wedge \sim ZF / CF$	无符号大于 / 小于

对有符号比较，不能只看 SF ：发生溢出时，结果的符号位不再反映真实大小。因此 $x < y$ 的条件是 $SF \wedge OF$ 。无符号比较则以 CF 为核心。

例：实现 `return x > y;`：

```
cmpq  %rsi, %rdi      # x - y
setg  %al              # al = (x > y)
movzbl %al, %eax      # 零扩展为 32 位返回值
ret
```

`setg` 只写 `%al`，所以要用 `movzbl` 清空 `%eax` 的其余位。写入 32 位寄存器 `%eax` 时，硬件也会把 `%rax` 的高 32 位清零。

4. 条件跳转与条件传送

4.1. 条件跳转 `jX`

`jmp` 无条件改变 `%rip`；`jX` 根据条件码决定是否跳转。其条件与 `setX` 对应，例如 `je`、`jne`、`jg`、`jle`、`ja`、`jb`。应特别区分：`g/l` 用于有符号比较，`a/b` 用于无符号比较。

将

```
long absdiff(long x, long y) {
    return x > y ? x - y : y - x;
}
```

翻译为分支形式：

```
absdiff:
    cmpq %rsi, %rdi
    jle .Lelse          # x <= y
    movq %rdi, %rax
    subq %rsi, %rax      # x - y
    ret
.Lelse:
    movq %rsi, %rax
    subq %rdi, %rax      # y - x
    ret
```

通用思路是：先求 !Test，若成立就跳往 Else；否则执行 then 部分，再跳过 else 部分到 Done。

4.2. 条件传送 cmovX

条件传送不改变控制流，而是当条件满足时执行 `Dest <- Src`。对应的 `cmovle`、`cmovg` 等条件也来自条件码。上例可以写成：

```
movq %rdi, %rax      # result = x - y
subq %rsi, %rax
movq %rsi, %rdx      # eval = y - x
subq %rdi, %rdx
cmpq %rsi, %rdi
cmovle %rdx, %rax     # x <= y 时选择 eval
ret
```

条件分支可能打断流水线；条件传送避免控制转移，适合两边计算都很简单的情形。但它必须先算出两个候选值，因此不适用于：

- 计算开销很大：两条路径都会被计算；
- 可能非法的表达式：如 `p ? *p : 0`，`p` 为零时仍会解引用；
- 有副作用的表达式：如 `x > 0 ? x *= 7 : x += 3`，不能让两边都执行。

5. 循环的汇编翻译

5.1. do-while

do-while 保证循环体至少执行一次：

```
do Body while (Test);
```

其基本的标签形式为：

```
loop:
    Body;
    if (Test) goto loop;
```

课件用 `pcount_do` 统计无符号整数中 1 的个数：每轮将最低位 `x & 1` 加到 `result`，再将 `x` 右移一位。其核心汇编为：

```
movl $0, %eax        # result = 0
.L2:
    movq %rdi, %rdx
    andl $1, %edx      # t = x & 1
    addq %rdx, %rax
```

```
shrq %rdi          # x >>= 1; 同时设置条件码
jne .L2            # x != 0 时继续
```

shrq 的结果正好决定是否继续，因此无须额外 test。注意：若初始 $x=0$ ，do-while 仍会执行一次；但本例第一次加的是 0，所以结果仍正确。

5.2. while

while (Test) Body 有两种常见翻译。

1) 跳到中间 (-Og 常见)：先无条件跳到测试，再由测试回到循环体。

```
goto test;
loop:
    Body;
test:
    if (Test) goto loop;
done:
```

1) 转换为 do-while (优化后常见)：先做一次入口检查；进入后在尾部检查。

```
if (!Test) goto done;
loop:
    Body;
    if (Test) goto loop;
done:
```

第二种消除了每次循环开头的跳转，但多了入口保护。选择何种形式由编译器优化策略和分支预测等因素决定。

5.3. for

for (Init; Test; Update) Body 与下列代码等价：

```
Init;
while (Test) {
    Body;
    Update;
}
```

因此 for 并不是独立的机器级结构，先按上述规则降为 while 即可。对于课件中的

```
for (i = 0; i < WSIZE; i++) {
    unsigned bit = (x >> i) & 1;
    result += bit;
}
```

初始化为 $i=0$ ；循环体先计算第 i 位并累加；更新 $i++$ 后重新测试 $i<WSIZE$ 。这里 $WSIZE=8*\text{sizeof}(\text{int})$ 是固定正数，所以编译器能知道初始的 $i=0$ 一定通过测试，入口保护可被优化掉。

6. switch 与跳转表

switch 能表达多路选择，也支持多个 case 共用代码、fall-through (不写 break 继续执行下一个 case) 和 default。当 case 值较密集时，编译器通常生成跳转表 (jump table)，用一次间接跳转完成分派。

6.1. 基本结构

若 x 的合法范围为 $0..n-1$ ，跳转表保存每个值对应代码块的地址：

```
jtab:  Targ0, Targ1, ..., Targ(n-1)
      goto *jtab[x]
```

每个地址在 x86-64 中占 8 字节，故索引时要乘 8。跳转前必须先检查范围，否则负数或过大的 x 会越界读取表。

课件例中 x 的表范围为 $0..6$ ：

```

cmpq $6, %rdi
ja    .L8          # 无符号 x > 6: 包括负数, 进入 default
jmp   *.L4(,%rdi,8) # 取地址 .L4 + x*8, 再间接跳转

```

这里使用无符号 `ja` 很关键：若 $x < 0$ ，把它解释为无符号数会非常大，同样满足 $x > 6$ ，从而安全地去 `default`。

6.2. 表项与共享代码块

跳转表可把不同的输入映射到同一目标：缺失的 `case`（如 0、4）映射到 `default`，`case 5` 与 `case 6` 共用一个代码块。`case 2` 的 `fall-through` 可理解为先完成除法，然后跳到 `case 3` 的合流处继续加法：

```

.L5:          # case 2
    movq %rsi, %rax
    cqto
    idivq %rcx    # rax = y / z
    jmp .L6
.L9:          # case 3
    movl $1, %eax    # w = 1
.L6:          # 合流处
    addq %rcx, %rax    # w += z
    ret

```

理解 `switch` 汇编时，先找范围检查与间接 `jmp`，再读取 `.rodata` 中各表项，最后顺着每个目标块追踪控制流。`case` 值稀疏时，跳转表会浪费空间，编译器更可能生成一串比较组成的决策树。

7. 小结

- 条件码记录最近算术或逻辑操作的结果状态；`cmp` 和 `test` 专门设置条件码而不保存计算结果。
- `setX` 将比较结果物化为 0/1；`jX` 通过改变 `%rip` 实现分支；`cmovX` 在不跳转的条件下选择值。
- 有符号比较要考虑 `OF`，无符号比较主要依赖 `CF`。不要混用 `jg/jl` 与 `ja/jb`。
- `do-while` 在末尾测试；`while` 可翻译为跳到中间或带入口保护的 `do-while`；`for` 可先化为 `while`。
- 稠密的 `switch` 通常使用跳转表，需先做范围检查；表索引按地址宽度缩放。